

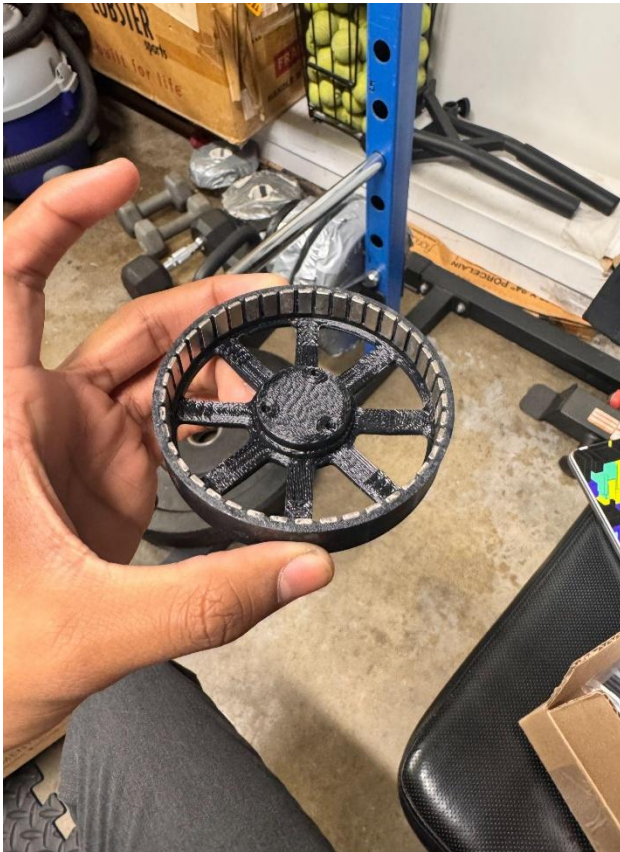
Aditya Verma

Windborne Systems Firmware Intern Application

1. Something I am proud of: BLDC Motor + Controller

I designed a 36-slot 42-pole 8010 outrunner, hand-wound all three phases (wye, 6 parallel strands, 26 AWG enameled), then built the FOC controller PCB to drive it: STM32G0, 10A/phase half-bridges, Kelvin-sensed shunts, independent gate edge tuning, and ADC sampling timed to the center of each 50 us PWM cycle to avoid switching noise. Reverse-polarity protection, overvoltage protection, regen braking, 40V LiPo input.

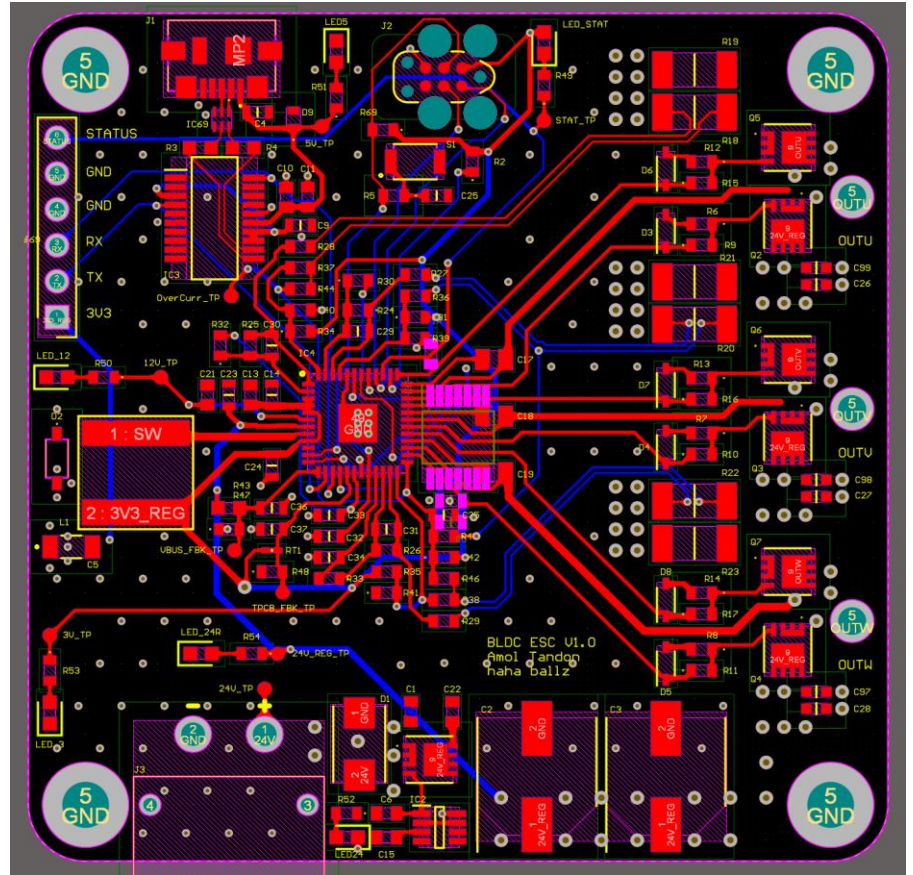
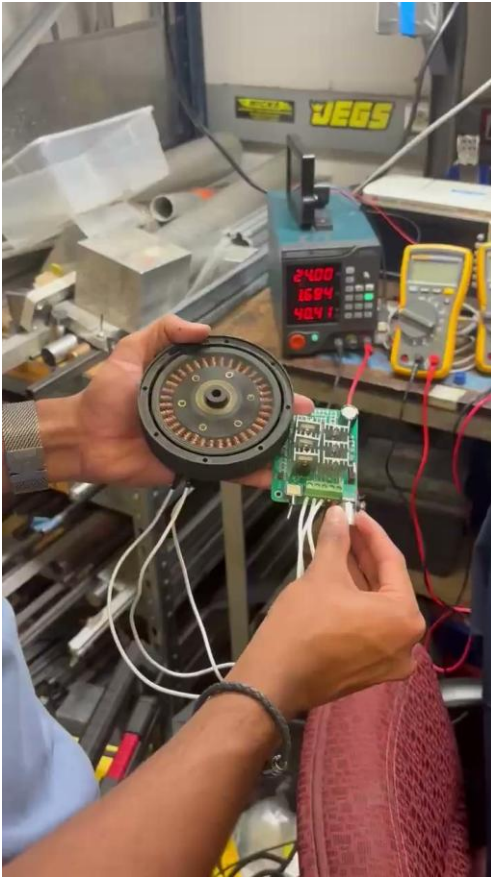
What I'm proud of: I had to understand the motor to make the controller work. Getting bootstrap cap sizing, dead time, and shoot-through risk right on a board I laid out myself taught me more about hardware/firmware co-design than any class I've taken. Also, the motor spins!



42 pole magnet array in rotor



wye-connected hand-wound stator



BLDC controller layout

2. My setup

VS Code with Cortex-Debug and a J-Link or ST-Link. I use STM32CubeIDE to configure peripherals visually, then edit the generated code in VS Code. Make or CMake in the terminal for everything else. I also started keeping a physical notebook after getting burned too many times by not being able to reproduce a result I half-remembered.

For pcb design I am constantly switching between Altium and KiCAD, depending on whether I am collaborating with someone on the project.

3. Debugging a PCB

First I'd need to figure out what kind of reset. I'd probe NRST and the supply rail with a scope in single-shot mode. If NRST falls before the supply droops, something is pulling the line. If the supply droops first, it's a power issue. If both look clean, I'd check the MCU's reset cause register, since STM32's log whether the last reset was a watchdog, software, power-on, or pin reset.

4. What Firmware engineers get wrong

Although I haven't had too much industry experience, I can say that the biggest thing I'd say is that most firmware engineers should start writing tests for embedded code until something breaks badly. Even just a host-compiled unit test for the state machine logic or a packet parser catches a surprising number of bugs before the hardware is involved at all. This tends to prevent damaging/expensive/dangerous failures before they have consequences.

5. Sketchy fix

On the Agrovolo drone project, our Teensy 4.0 flight controller was reading GPS over UART and we were getting occasional corrupted frames that caused the position-hold loop to briefly see a bogus location and twitch the motors. The right fix was a proper parser with checksum validation and a frame-drop counter feeding into the control loop. What we actually used for the first flight test was a median filter over the last three GPS readings with a hard clamp: if the new reading differed from the median by more than a threshold, discard it and reuse the last good value. No checksum, no root cause fix on the UART noise, no stress right?

Well, the band-aid fix ended up working. We squeezed three stable test flights out of the drone before it ran out of juice. I'm a little proud of it because it took about 10 minutes to write and freed up the whole team for the day, but at the same time I'm also aware that if the GPS had been consistently wrong rather than noisy, the filter would have happily passed through bad data and we'd have had a more interesting afternoon.